MODULE 1: INTRODUCTION TO MACHINE LEARNING

1. WHAT IS MACHINE LEARNING ?

Machine Learning (ML) is a field of Artificial Intelligence that enables computers to learn from data and improve their performance on a task without being explicitly programmed.

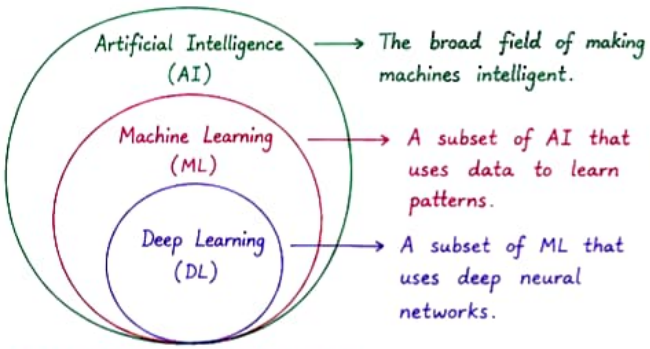
> In simple words : ML = Learning from data to make better decisions / predictions.

2. DEEPLEARNING.AI DEFINITION

> "ML is the study of computer algorithms that improve automatically through experience."
> — Andrew Ng, DeepLearning.AI

It is about building systems that can learn from data and generalize to new, unseen data.

3. RELATIONSHIP BETWEEN AI, ML & DEEP LEARNING



4. WHY MACHINE LEARNING ?

- Huge amount of data is available.
- Hard to write rules for complex problems.
- ML models can adapt and improve with experience.
- Useful in many real-world applications.

Examples : Email spam filter, Recommendation systems, Self driving cars, Speech recognition, Fraud detection etc.

5. TYPES OF MACHINE LEARNING

1. Supervised Learning	Learn from labeled data. Input and output (target) are known.	Examples : • Email spam (spam / not spam) • House price prediction
2. Unsupervised Learning	Learn from unlabeled data. Find hidden patterns or structure.	Examples : • Customer segmentation • Grouping similar documents
3. Reinforcement Learning	Learn by interacting with an environment and getting rewards.	Examples : • Game playing • Robot navigation

Other important terms :

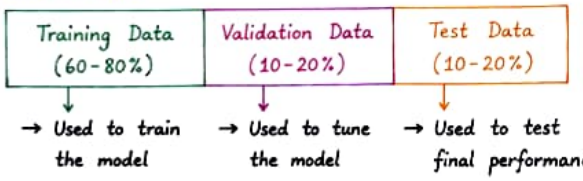
- Feature (X) : Input variable used for prediction.
- Target / Label (Y) : Output we want to predict.
- Model : : The system that learns from data.
- Training : : Learning the model using training data.
- Prediction : : Using the model to predict new data.
- Evaluation : : Checking how good the model is.

☆

6. DATA IN MACHINE LEARNING

- Data is the backbone of ML.
- Quality data → Good model
- Types of data :
  1. Numerical (e.g., age, salary)
  2. Categorical (e.g., gender, color)
  3. Text (e.g., reviews, emails)
  4. Images, Audio, Video

Dataset split :



7. WORKFLOW OF ML PROJECT

1. Problem Understanding → Define the problem clearly.
2. Data Collection → Gather relevant data.
3. Data Preparation → Clean and preprocess data.
4. Exploratory Data Analysis (EDA) → Understand the data.
5. Model Building → Choose algorithm and train.
6. Model Evaluation → Check performance.
7. Hyperparameter Tuning → Improve model.
8. Prediction / Deployment → Use model in real world.

8. COMMON ML TASKS

- Classification → Predict a category / class.
  e.g., Spam detection, Disease prediction
- Regression → Predict a continuous value.
  e.g., House price, Temperature
- Clustering → Group similar data points.
  e.g., Customer segmentation
- Association Rule Mining → Find relationships.
  e.g., Market basket analysis

9. SOME COMMON ML ALGORITHMS

For Classification :

- Linear Regression
- Logistic Regression
- Decision Tree
- Random Forest
- K-Nearest Neighbors (KNN)
- Support Vector Machine (SVM)

For Regression :

- Linear Regression
- Polynomial Regression
- Decision Tree Regressor
- Random Forest Regressor
- Support Vector Regression (SVR)

10. REAL WORLD APPLICATIONS

- Healthcare : Disease prediction, Medical imaging
- Finance : Fraud detection, Stock prediction
- E-commerce : Product recommendation
- Transportation : Traffic prediction, Self driving cars
- NLP : Chatbots, Language translation
- Others : Weather forecast, Face recognition

KEY TAKEAWAY

ML allows machines to learn from data, find patterns and make intelligent decisions without being explicitly programmed. ☆

QUICK SUMMARY

Data (input) → Machine Learning (learn patterns) → Model (trained) → Prediction (output) → Better Decisions (value)

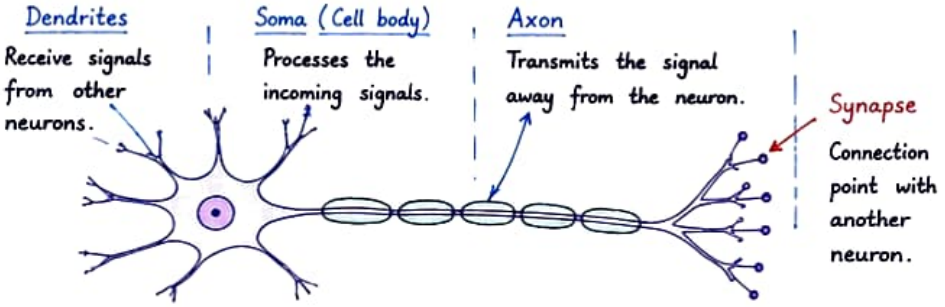
Learn, Improve and Adapt! ☺

Review of Neural Networks

1. Model of a Biological Neuron

What is a Neuron?

A neuron is a specialized cell in the nervous system that receives, processes and transmits information.



Signal flow : Dendrites → Soma → Axon → Synapse → Next neuron

How does a biological neuron work?

- Dendrites receive signals.
- Soma processes the signals.
- If the signal is strong enough, the neuron 'fires'.
- The signal is transmitted through the axon to other neurons via synapses.

Biological Neuron vs Artificial Neuron

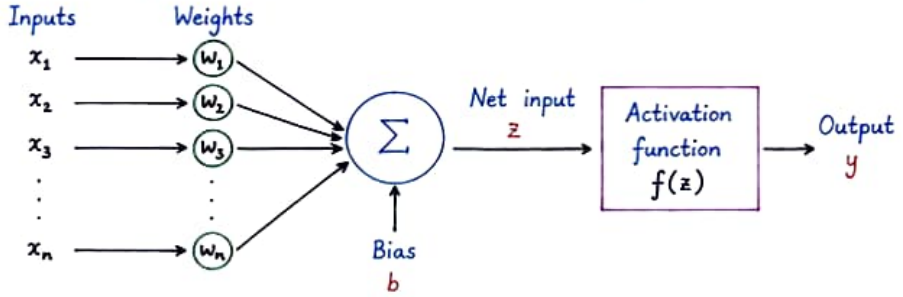
Biological Neuron		Artificial Neuron
Dendrites	⟷	Inputs
Synapses	⟷	Weights
Soma (cell body)	⟷	Summation / Processing
Axon	⟷	Output
Firing of neuron	⟷	Activation / Output
Strength of synapse	⟷	Weight value

From Biological Neuron to Artificial Neuron

Biological Neuron → Artificial Neuron

Artificial neuron is a mathematical model inspired by the biological neuron.

Artificial Neuron (Mathematical Model)



Net input (weighted sum + bias)	Output
$z = \sum_{i=1}^{n} w_i x_i + b$	$y = f(z)$

Explanation of terms

- Input (x) : Information provided to the neuron.
- Weight (w) : Importance of each input.
- Bias (b) : Extra adjustable value (shifts the decision boundary).
- Net input (z) : Total weighted input to the neuron.
- Activation function (f) : Decides the output.
- Output (y) : Final result produced by the neuron.

Simple Numerical Example

Suppose,

$x_1 = 2$, $x_2 = 3$ $w_1 = 0.5$, $w_2 = 0.2$ $b = 1$

$z = (2 \times 0.5) + (3 \times 0.2) + 1 = 1 + 0.6 + 1 = 2.6$

$y = f(2.6)$ → depends on activation function.

Key Points to Remember

- ☆ A neuron is the basic unit of the nervous system.
- ☆ Dendrites → Soma → Axon → Synapse.
- ☆ Artificial neuron receives inputs, processes them and produces an output.
- ☆ Learning in a neural network mainly means learning appropriate weights.
- ☆ More than one neuron connected together forms a Neural Network.

Analogy

Deciding "Should I go to college today?"

- Inputs : Is there an exam? Is it raining? Is attendance low?
- Weights : Importance of each factor
- Neuron : Calculates total + bias → activation → Go / Don't Go

McCulloch-Pitts Neuron (M-P Neuron)

1. INTRODUCTION

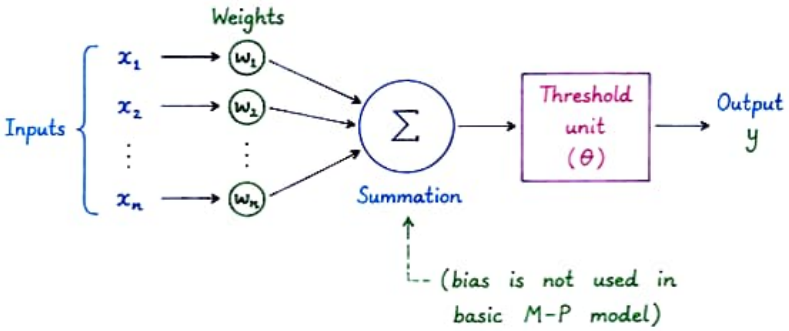
- Proposed by Warren McCulloch and Walter Pitts in 1943.
- It is one of the earliest mathematical models of an artificial neuron.
- It is a simple threshold logic unit.
- The neuron produces only two possible outputs : 0 (does not fire) or 1 (fires).

2. BASIC IDEA

Inputs $x_1, x_2, \ldots, x_n$ → Combine inputs → Compare with threshold θ → Output $y = 0$ or 1

- If total input ≥ threshold (θ) → neuron fires → $y = 1$
- If total input < threshold (θ) → neuron does not fire → $y = 0$

3. STRUCTURE OF M-P NEURON



4. MATHEMATICAL MODEL

Let,

x_i = input (usually binary 0 or 1)

w_i = weight associated with input x_i

θ = threshold

n = number of inputs

Net input (S) = $S = \sum_{i=1}^{n} w_i x_i$

Output (y) = $\begin{cases} 1, & \text{if } S \geq \theta \\ 0, & \text{if } S < \theta \end{cases}$

5. ASSUMPTIONS

- Inputs are binary (0 or 1).
- Output is binary (0 or 1).
- Weights and threshold are fixed (not learned).
- Inputs can be excitatory (promote firing) or inhibitory (prevent firing).
- No learning rule in basic M-P neuron.

Excitatory (+) help neuron to reach threshold

Inhibitory (−) opposes neuron from reaching threshold

6. EXCITATORY AND INHIBITORY INPUTS

- Excitatory input has positive weight (+1).
- Inhibitory input has negative weight (−1).
- Total input is the algebraic sum of all inputs.

S = (sum of excitatory inputs) − (sum of inhibitory inputs)

If $S \geq \theta$ → neuron fires ($y = 1$)

If $S < \theta$ → neuron does not fire ($y = 0$)

7. IMPLEMENTATION OF LOGIC GATES

(a) AND Gate

Choose $w_1 = 1$, $w_2 = 1$, $\theta = 2$

x_1	x_2	$w_1x_1 + w_2x_2$	Output (y)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	2 ≥ 2	1

Fires only when both inputs are 1.

(b) OR Gate

Choose $w_1 = 1$, $w_2 = 1$, $\theta = 1$

x_1	x_2	$w_1x_1 + w_2x_2$	Output (y)
0	0	0 < 1	0
0	1	1 ≥ 1	1
1	0	1 ≥ 1	1
1	1	2 ≥ 1	1

Fires when at least one input is 1.

(c) NOT Gate

Use one inhibitory input.

Choose $w = -1$, $\theta = 0$

x	$w\,x$	Output (y)
0	0 ≥ 0	1
1	−1 < 0	0

Fires when input is 0.

8. M-P NEURON VS PERCEPTRON

M-P Neuron	Perceptron
• Early model (1943).	• Proposed by Rosenblatt (1958).
• Inputs usually binary.	• Inputs are numerical.
• Output is binary.	• Output can be binary.
• No learning rule.	• Has learning rule to update weights.
• Weights & threshold are fixed.	• Weights are learned.
• Implements logic gates.	• Used for classification.

9. KEY TAKEAWAY ☆

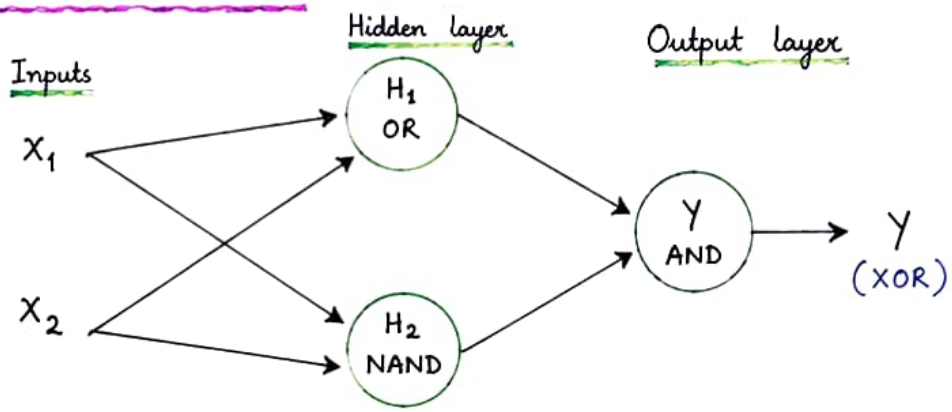
McCulloch-Pitts neuron is a binary threshold logic unit. It forms the foundation for later neural network models. ☆

Simple rule, Powerful idea ! ☺

XOR using MP Neurons

XOR = (X_1 OR X_2) AND (X_1 NAND X_2)

1. Network Structure



$H_1 = X_1$ OR X_2

$H_2 = X_1$ NAND X_2

$Y = H_1$ AND H_2 = XOR

2. Weights and Thresholds

(i) Neuron H_1 (OR)

$w_1 = 1$, $w_2 = 1$, $\theta = 1$

x_1	x_2	net input	output
0	0	0	0
0	1	1	1
1	0	1	1
1	1	2	1

net = $x_1 + x_2$

output = 1 if net ≥ 1

(ii) Neuron H_2 (NAND)

$w_1 = -1$, $w_2 = -1$, $\theta = -1$

x_1	x_2	net input	output
0	0	0	1
0	1	−1	1
1	0	−1	1
1	1	−2	0

net = $-x_1 - x_2$

output = 1 if net ≥ -1

(iii) Neuron Y (AND)

$w_{H_1} = 1$, $w_{H_2} = 1$, $\theta = 2$

H_1	H_2	net input	output
0	0	0	0
0	1	1	0
1	0	1	0
1	1	2	1

net = $H_1 + H_2$

output = 1 if net ≥ 2

3. Truth Table

x_1	x_2	$H_1 = x_1$ OR x_2	$H_2 = x_1$ NAND x_2	$Y = H_1$ AND H_2 (XOR)
0	0	0	1	0
0	1	1	1	1
1	0	1	1	1
1	1	1	0	0

Results:

Y = 1 for (0,1) and (1,0)

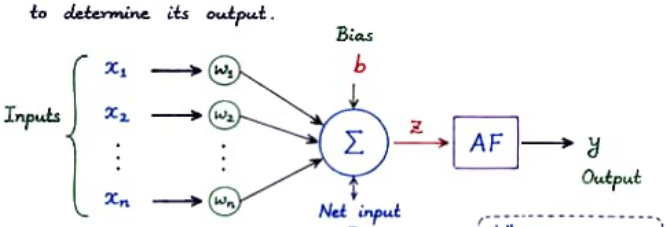
Y = 0 for (0,0) and (1,1)

Hence, Y is XOR.

✳ ACTIVATION FUNCTIONS ✳

1. What is an Activation Function ?

An activation function (AF) is a mathematical function applied to the net input of a neuron to determine its output.



$$z = \sum_{i=1}^{n} w_i x_i + b \qquad y = f(z)$$

Where,
- x_i : input
- w_i : weight
- b : bias
- z : net input
- f : activation function
- y : output

2. Why do we need Activation Functions ?

- Without activation function ($f(z) = z$), no matter how many layers we stack, the whole network behaves like a linear function.
- Activation functions introduce non-linearity into the network, allowing it to learn complex relationships.
- They also control the range of the output and help in decision making.

3. Characteristics of a Good Activation Function

- Non-linear (except output layer in regression)
- Differentiable (for learning using gradient descent)
- Computationally efficient
- Output in a useful range

Key Takeaway

Activation functions introduce non-linearity into neural networks and make them powerful learners of complex patterns. The choice of AF depends on the problem and the layer in which it is used. ☺

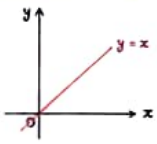
4. Types of Activation Functions

- Linear (Identity) Activation Function
- Non – Linear Activation Functions
  - Binary Step (Threshold)
  - Sigmoid (Logistic)
  - Tanh (Hyperbolic Tangent)
  - ReLU (Rectified Linear Unit)
  - Leaky ReLU
  - ELU (Exponential Linear Unit)
  - Softmax (for Multi-class)

5. Linear (Identity) Activation Function

$$f(x) = x$$

- Also called Identity function.
- Output is same as input.
- Range : $(-\infty, \infty)$
- Derivative : $f'(x) = 1$
- Graph : Straight line ($y = x$)



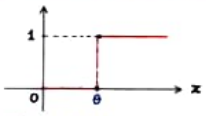
Used in :
- Regression problems (especially output layer)
- Predicting real-valued quantities.

Advantages	Disadvantages
✓ Simple	✗ No non-linearity
✓ Computationally cheap	✗ Not suitable for hidden layers
✓ Suitable for regression	✗ Cannot solve complex non-linear problems

6. Non – Linear Activation Functions

(A) Binary Step (Threshold)

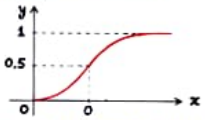
$$f(x) = \begin{cases} 1, & x \geq \theta \\ 0, & x < \theta \end{cases}$$



Example : $\theta = 2$
$x = 3 \rightarrow y = 1$
$x = 1 \rightarrow y = 0$

(B) Sigmoid (Logistic)

$$f(x) = \frac{1}{1 + e^{-x}}$$



Example : $x = 2 \rightarrow f(x) \approx 0.881$

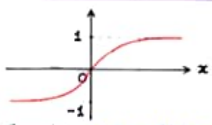
Range : $(0, 1)$

Derivative : $f'(x) = f(x)[1 - f(x)]$

Used in : Binary classification (output layer).

(C) Tanh (Hyperbolic Tangent)

$$f(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$



Example : $x = 2 \rightarrow f(x) \approx 0.964$
$x = -2 \rightarrow f(x) \approx -0.964$

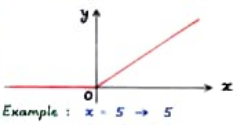
Range : $(-1, 1)$

Derivative : $f'(x) = 1 - f^2(x)$

Used in : Hidden layers.

(D) ReLU (Rectified Linear Unit)

$$f(z) = \max(0, x)$$



Example : $x = 5 \rightarrow 5$
$x = -3 \rightarrow 0$

Range : $[0, \infty)$

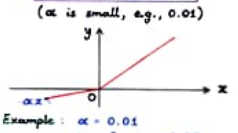
Derivative : $f'(x) = \begin{cases} 1, & x > 0 \\ 0, & x < 0 \end{cases}$
(not defined at $x = 0$)

Used in : Hidden layers.

(E) Leaky ReLU

$$f(x) = \begin{cases} x, & x \geq 0 \\ \alpha x, & x < 0 \end{cases}$$

(α is small, e.g., 0.01)



Example : $\alpha = 0.01$
$x = -5 \rightarrow -0.05$
$x = 5 \rightarrow 5$

Range : $(-\infty, \infty)$

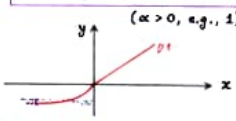
Derivative : $f'(x) = \begin{cases} 1, & x > 0 \\ \alpha, & x < 0 \end{cases}$

Used in : McCulloch-Pitts neuron, Perceptron.

(F) ELU (Exponential Linear Unit)

$$f(x) = \begin{cases} x, & x \geq 0 \\ \alpha(e^{x} - 1), & x < 0 \end{cases}$$

($\alpha > 0$, e.g., 1)



Example : $\alpha = 1$
$x = -1 \rightarrow -e^{-1} + 1 = -0.632$
$x = 2 \rightarrow 2$

Range : $(-\alpha, \infty)$

Derivative : $f'(x) = \begin{cases} 1, & x > 0 \\ \alpha e^{x}, & x < 0 \end{cases}$

Used in : Hidden layers.

(G) Softmax (for Multi-class)

$$f(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}} \qquad (i = 1, 2, \ldots, K)$$

$$\sum_{i=1}^{K} f(z_i) = 1$$

Example : For 3 classes,
$z = [2, 1, 0]$
$e^{z} = [7.389, 2.718, 1]$
Sum = 11.107
Softmax = $[0.665, 0.245, 0.090]$

Range : $(0, 1)$ and sum = 1

Used in : Output layer for multi-class classification.

7. Derivative (in general)

Derivative of a function $f(x)$ is $f'(x) = \frac{dy}{dx}$

It tells how much output changes with respect to input input.

Important for Gradient descent & Backpropagation.

8. Monotonic Function

A function is monotonic if it is either always increasing or always decreasing.
- Increasing : $f'(x) \geq 0$
- Decreasing : $f'(x) \leq 0$

Quick Summary

Function	Formula	Range	Graph Shape	Derivative	Used In
Linear	$f(x) = x$	$(-\infty, \infty)$	Straight line	1	Regression (output)
Step	0/1 (threshold)	$\{0, 1\}$	Step	0 (except at θ)	M-P, Perceptron
Sigmoid	$\frac{1}{1+e^{-x}}$	$(0, 1)$	S-curve	$f(x)(1-f(x))$	Binary class (output)
Tanh	$\frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$	$(-1, 1)$	S-curve (centered)	$1 - f^2(x)$	Hidden layers
ReLU	$\max(0, x)$	$[0, \infty)$	Half line	1 ($x>0$), 0 ($x<0$)	Hidden layers
Leaky ReLU	x or αx	$(-\infty, \infty)$	Line with leak	1 or α	Hidden layers
ELU	x or $\alpha(e^{x}-1)$	$(-\alpha, \infty)$	Exp. curve + line	1 or αe^{x}	Hidden layers
Softmax	$\frac{e^{z_i}}{\sum_j e^{z_j}}$	$(0, 1)$, sum = 1	Probability dist.	Complex	Multi-class (output)

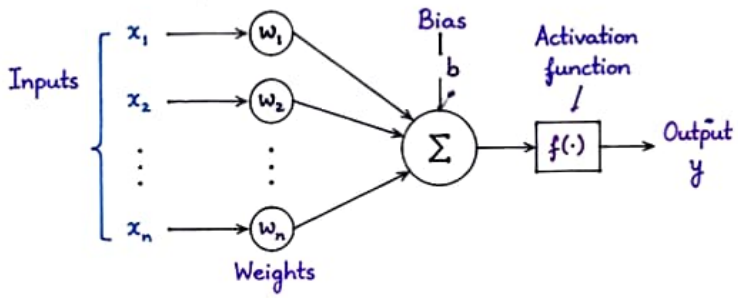
Remember

- ✓ Linear AF → simple, no non-linearity.
- ✓ Non-linear AF → powerful, learn complex relationships.
- ✓ Sigmoid → 0 to 1
- ✓ Tanh → -1 to 1
- ✓ ReLU → fast, widely used
- ✓ Softmax → probabilities for multi-class
- ✓ Derivative → essential for learning

☺

1. PERCEPTRON

Perceptron is the simplest form of artificial neuron. It is a linear binary classifier.



Weighted Sum :

$$net = \sum_{i=1}^{n} w_i x_i + b$$

Activation function (step function) :

$$f(net) = \begin{cases} 1 & \text{if } net \geq 0 \\ 0 & \text{if } net < 0 \end{cases}$$

Output $y = f(net) \approx \{0, 1\}$

Interpretation :

- $y = 1 \longrightarrow$ positive class
- $y = 0 \longrightarrow$ negative class

2. PERCEPTRON LEARNING ALGORİTHM

Goal : Find weights (w) and bias (b) such that the perceptron correctly classifies all training patterns.

Algorithm :

① Initialize weights (w_i) and bias (b) to small values (often 0).

② For each training example $(x_1, x_2, \ldots, x_n)$, y
- Compute $net = \sum_{i=1}^{n} w_i x_i + b$
- Compute output $\bar{y} = f(net)$
- Compute error $e = y - \bar{y}$
- Update weights and bias (only if $e \neq 0$)

$$w_i \text{ (new)} = w_i \text{ (old)} + \eta (y - \bar{y}) x_i \quad \text{for } i = 1 \ldots n$$

$$b \text{ (new)} = b \text{ (old)} + \eta (y - \bar{y})$$

where η = learning rate $(0 < \eta \leq 1)$

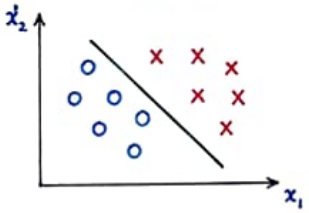
③ Repeat step 2 until no error occurs for all training examples or until maximum epochs reached.

Note : Weight update occurs only when the perceptron makes a mistake.

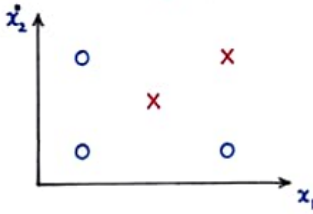
3. CONVERGENCE OF PERCEPTRON

Perceptron Learning Algorithm is guaranteed to converge (i.e., reach a solution in finite steps) if and only if the training data is linearly separable.

: Linearly Separable



Not Linearly Separable (e.g., XOR)



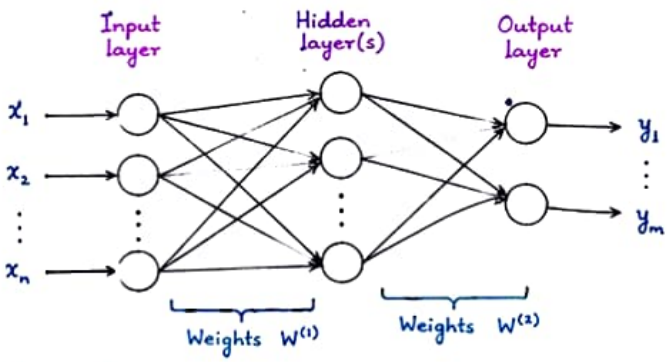
If data is linearly separable → algorithm converges.
If data is not linearly separable → algorithm does not converge.

Limitations of single perceptron :

- Can solve only linearly separable problems.
- Cannot solve problems like XOR.

4. MULTİLAYER PERCEPTRON (MLP)

MLP is a feedforward neural network with one or more hidden layers.



Key points :

- Hidden layer enables the network to learn complex, non-linear decision boundaries.
- Uses non-linear activation functions (sigmoid, tanh, ReLU, etc.) in hidden layers.
- Trained using Backpropagation with gradient descent.

Advantages of MLP :

✓ Can model non-linear relationships.
✓ Can solve problems like XOR.
✓ More powerful than single perceptron.

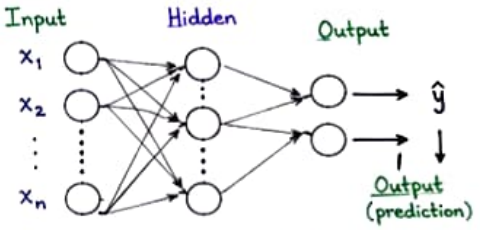
1. BACK PROPAGATION

* Backpropagation is an algorithm used to train neural networks.
* It works in two passes:

(1) Forward pass → compute output

(2) Backward pass → compute error gradients.

Goal: find how much each weight contributed to the error and update the weights.



$$\text{Error} = \frac{1}{2}\sum (y - \hat{y})^2$$

Steps:

1. Forward pass: compute output $\hat{y}$ from input x.
2. Compute error E.
3. Backward pass: from output layer to input layer, compute gradient of E w.r.t. each weight.
4. Update weights using gradient descent.

Weight update: $w_{new} = w_{old} - \eta \frac{\partial E}{\partial w}$

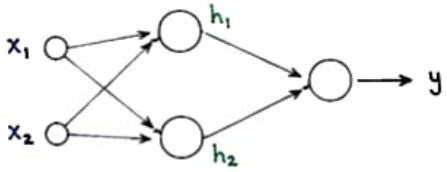
(η = learning rate)

2. LEARNING XOR

* XOR truth table

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

* A single perceptron cannot learn XOR (not linearly separable).
* Use a neural network with one hidden layer.



Example of what hidden neurons can learn:

x_1	x_2	OR	NAND	XOR (= OR AND NAND)
0	0	0	1	0
0	1	1	1	1
1	0	1	1	1
1	1	1	0	0

Training (using backprop + gradient descent):

Random weights → Forward pass → Compute error

Update weights ← Compute gradients ← Update weights

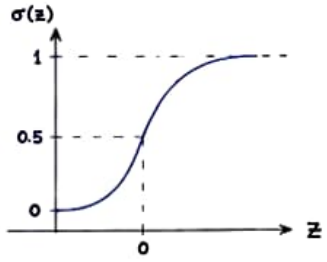
Repeat many times until error is small.

3. SIGMOID NEURONS

* A sigmoid neuron uses sigmoid activation function.

Sigmoid function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



* Output range: $(0, 1)$
* At $z = 0$, $\sigma(0) = 0.5$
* Derivative:

$$\sigma'(z) = \sigma(z)\,(1 - \sigma(z))$$

Neuron computation:

- $z = w_1 x_1 + w_2 x_2 + \ldots + w_n x_n + b$
- $a = \sigma(z)$ (output)

Example:

Let $x_1 = 2$, $x_2 = 3$, $w_1 = 0.5$, $w_2 = -0.4$, $b = 0.1$

$z = (0.5)(2) + (-0.4)(3) + 0.1 = 0.1$

$a = \sigma(0.1) = \frac{1}{1 + e^{-0.1}} \approx 0.525$

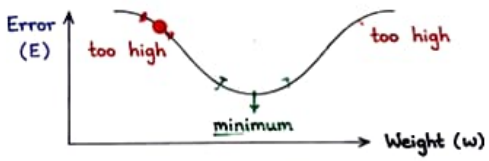
4. GRADIENT DESCENT

* Gradient Descent is an optimization technique used to minimize error.
* It updates the weights in the direction opposite to the gradient.

$$w_{new} = w_{old} - \eta \frac{\partial E}{\partial w}$$

(η = learning rate)

Intuition (on a hill):



* Choose learning rate carefully:
  - Too large η → overshoot
  - Too small η → slow learning

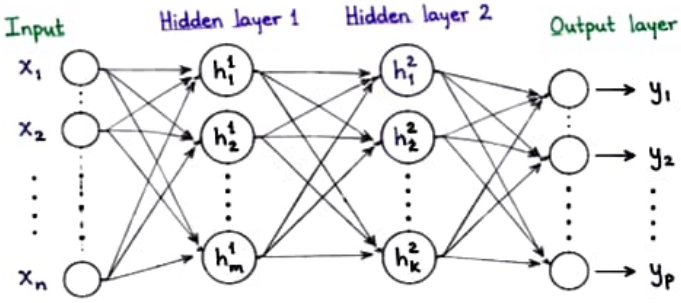
Example:

$w = 0.5$, $\frac{\partial E}{\partial w} = 0.3$, $\eta = 0.1$

$w_{new} = 0.5 - (0.1)(0.3) = 0.47$

5. FEED FORWARD NEURAL NETWORKS

* In a feed forward neural network, information flows only in one direction: Input → Hidden layers → Output. There are no loops or cycles.



Working (for one neuron):

1. Weighted sum: $z = w_1 x_1 + w_2 x_2 + \ldots + w_n x_n + b$
2. Activation: $a = f(z)$ (e.g., sigmoid, ReLU)
3. Output is passed to next layer.

Applications: classification, regression, pattern recognition, forecasting, and many more.

Typical Flow:

Input → Forward pass → Prediction → Compute error → Backpropagation → Update weights